CS1004 – Object Oriented Programming

Lecture # 23
Wednesday, May 11, 2022
Spring 2022
FAST – NUCES, Faisalabad Campus

Muhammad Yousaf

Today's Lecture

- Polymorphism in C++
- Pointers to derived classes
- Introduction to virtual functions

Pointers to Objects

```
class Test {
private:
int n;
public:
void in()
{
                cout << "Enter number! ";
                cin >> n;
}
void out()
{
                cout << "The value of n =" << n;
}
};
int main()
{
Test *ptr;
ptr = new Test;
ptr->in();
ptr->out();
return 0;
}
```

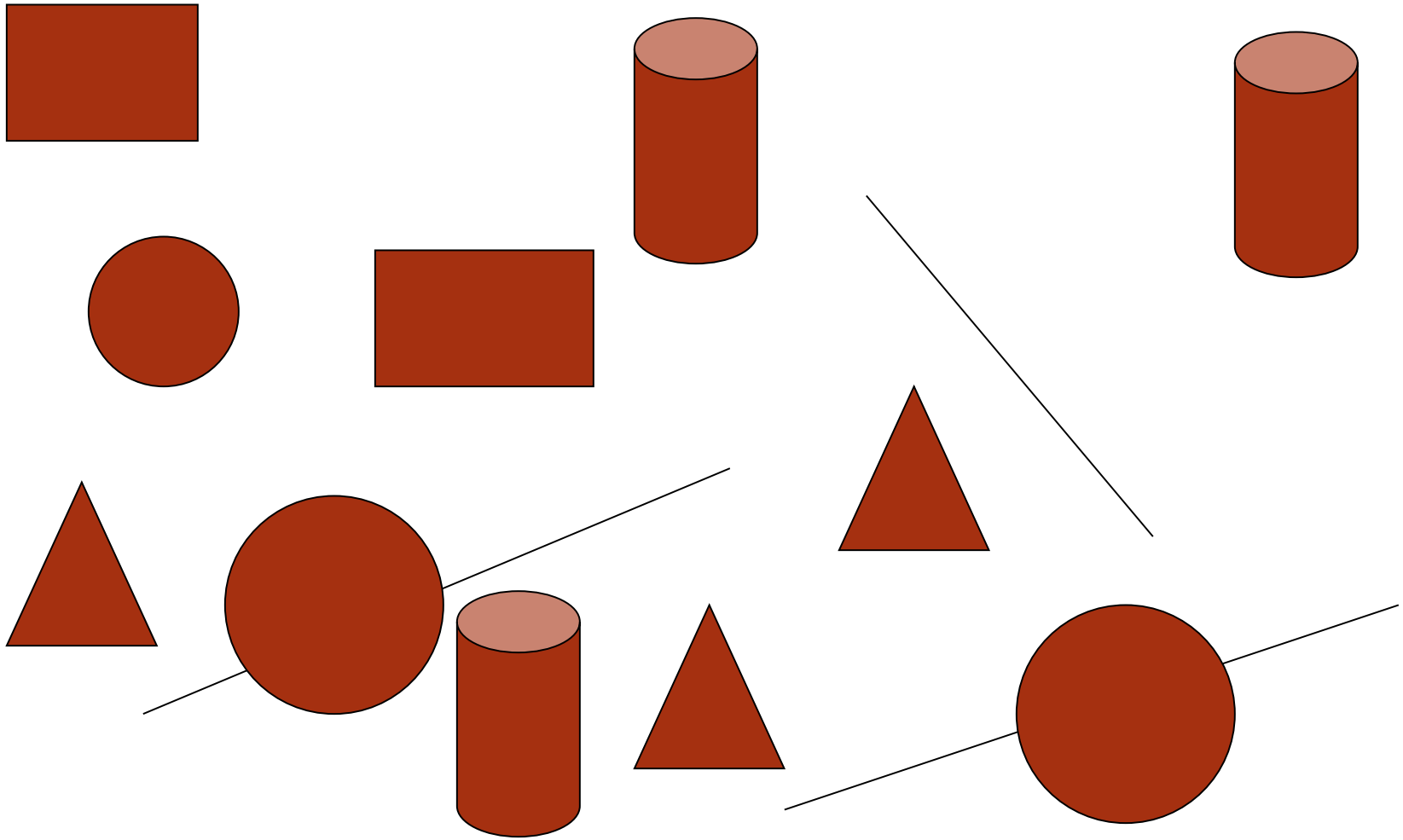
Polymorphism

- The Greek word polymorphism means one name, many forms
- Typically, polymorphism occurs when there is a hierarchy of classes and they are related by inheritance
- C++ polymorphism means that a call to a member function will cause a different function to be executed depending on the type of object that invokes the function

- **Static polymorphism:** It can be achieved by using overloading. It is defined at compilation time

- **Dynamic polymorphism:** It can be implemented by using inheritance. It is implemented at runtime

Graphics Drawing Software



Graphics Drawing Software Classes

- **Line**
  - **Properties:** X-Y Coordinates, Length, Color
  - **Actions:** Draw Function, Change Color Function, Get Area Function
- **Circle**
  - **Properties:** X-Y Coordinates, Radius, Color
  - **Actions:** Draw Function, Change Color Function, Get Area Function
- **Rectangle**
  - **Properties:** X-Y Coordinates, Width, Height, Color
  - **Actions:** Draw Function, Change Color Function, Get Area Function
- **Cylinder**
  - **Properties:** X-Y Coordinates, Radius, Height, Color
  - **Actions:** Draw Function, Change Color Function, Get Area Function
- **Triangle**
  - **Properties:** X-Y Coordinates, Length, Width, Color
  - **Actions:** Draw Function, Change Color Function, Get Area Function

```
class Line
{
protected:
int x, y;
public:
Line(int, int);
void draw(void);
int GetArea(void);
};
Line::Line(int a, int b)
{
x = a;
y = b;
}
void Line::draw(void)
{
cout << "\n Line Drawing code" ;
}
int Line::GetArea(void)
{
cout << "\nLine Area " ;
}
```

```
class Rectangle : public Line
{
protected:
int Width, Height;
public:
Rectangle(int, int, int, int);
void draw(void);
int GetArea(void);
};
Rectangle::Rectangle(int a, int b, int c, int d) : Line(a, b)
{
Width = c; Height = d;
}
void Rectangle::draw(void)
{
cout << "\n Rectangle drawing code" ;
}
int Rectangle::GetArea(void)
{
cout << "\n Rectangle area code" ;
}
```

```
class Circle : public Line
{
protected:
int radius;
public:
Circle(int, int, int);
void draw(void);
int GetArea(void);
};
Circle::Circle(int a, int b, int c) : Line(a, b)
{
radius = c;
}
void Circle::draw(void)
{
cout << "\n Circle drawing code ";
}
int Circle::GetArea(void)
{
cout << "\n Circle area code ";
}
```

```
int main(void)
{
Triangle t1(3, 4, 5, 19);
Circle c1(3, 4, 5);
Rectangle r1(3, 4, 10, 20);
Cylinder c2(3, 4, 5, 10);

t1.draw();
cout << "The area is " << t1.GetArea();

c1.draw();
cout << "The area is " << c1.GetArea();

r1.draw();
cout << "The area is " << r1.GetArea();

c2.draw();
cout << "The area is " << c2.GetArea();
return 0;
}
```

Pointer to derived class

- C++ allows base class pointers to point to derived class objects.
- Allows us to have –
  - class base { … };
  - class derived : public base { … };
- Then we can write –

```
base *p1;
derived d_obj;
p1 = &d_obj;
base *p2 = new derived;
```

Pointers to Derived Classes (contd.)

- Using a base class pointer (pointing to a derived class object) we can access only those members of the derived object that were inherited from the base
- This is because the base pointer has knowledge only of the base class
- It knows nothing about the members added by the derived class

Pointer of base class pointing to child class

```
class base {
public:
void show()
{
        cout << “base\n”;
}
};

class derived : public base {
public:
void show()
{
        cout << “derived\n”;
}
};
```

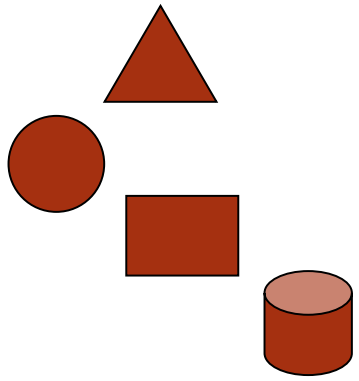
```
int main()
{
base b1;
b1.show(); // base
derived d1;
d1.show(); // derived

base *pb = &b1;   // base
pb->show();
pb = &d1;         // base
pb->show();
return 0;
}
```

Pointers to Derived Classes

- With the help of pointers to derived classes, we can create an array of base class objects, and that array can hold objects of different derived classes

```
Line *p[4];
p[0] = new Triangle(3, 4, 5, 19);
p[1] = new Circle(3, 4, 5);
p[2] = new Rectangle(3, 4, 10, 20);
p[3] = new Cylinder(3, 4, 5, 10);
for (int loop = 0; loop < 4; loop++)
{
        p[loop]->draw();
        cout << "The area is " << p[loop]->GetArea();
}
```



Pointers to Derived Classes (contd.)

- While it allows a base class pointer to point to a derived object, **the reverse is not true**

```
base b1;
derived *pd = &b1; // compiler error
```

Base class pointer pointing to child classes

```
class A {
public:
void show()
{
            cout << "Parent class A ";
}
};
class B : public A{
public:
void show()
{
            cout << "Parent class B ";
}
};
class C : public A{
public:
void show()
{
            cout << "Parent class C ";
}
};
```

```
int main()
{
A obj1;
B obj2;
C obj3;
A *Ptr;
Ptr = &obj1;
Ptr->show();
Ptr = &obj2;
Ptr->show();
Ptr = &obj3;
Ptr->show();
return 0;
}
```

Introduction to Virtual Functions

- A virtual function is a member function that is declared within a base class and **redefined (called overriding)** by a derived class
- It implements the "**one interface, multiple methods**" philosophy that **underlies polymorphism**
- The keyword **virtual** is used to designate a member function as virtual
- Supports run-time polymorphism with the help of base class pointers

Introduction to Virtual Functions (contd.)

```
class base {
public:
virtual void show() {
        cout << "\nBase Class" <<
endl;
}
};
class derived :public base {
public:
void show() {
        cout << "\nDerived Class"
                << endl;
}
};
```

```
int main()
{
base b1;
b1.show();
derived d1;
d1.show();

base *pb = &b1;
pb->show();

pb = &d1;
pb->show();
return 0;
}
```

Introduction to Virtual Functions (contd.)

```
class base {
public:
virtual void show() {
            cout << "\nBase Class" << endl;
}
};
class d1 :public base {
public:
void show() {
            cout << "\nDerived-1 Class"
                        << endl;
}
};
class d2 :public base {
public:
void show() {
            cout << "\nDerived-2 Class"
                        << endl;
}
};
```

```
int main()
{
base *pb;
d1 od1;
d2 od2;
int n;
cin >> n;
if (n % 2)
        pb = &od1;
else
        pb = &od2;
pb->show(); // guess what ??
}
```

Dynamic Binding

Run-time polymorphism

Static vs. Dynamic Binding

- There are two type of bindings in C++

Static Binding	Dynamic binding
Happens at compile time. Also called early binding	Happens at runtime. Also called late binding
Function definition and function call are linked during compile time	Function calls are not resolved until runtime. So they are not bound until runtime
Static binding happens when all the information to call a function is available at compile time	Dynamic binding happens when all information needed for a function call cannot be determined at compile time
Static binding can be achieved during normal function call, function overloading and operator overloading	Dynamic binding can be achieved using the virtual functions
Since all information needed to call a function is available before runtime, so static binding executes faster	Unlike static binding, a function call is not resolved until runtime for later binding and this results in somewhat slower execution

Static vs. Dynamic Binding

- The major advantage of dynamic binding is that it is flexible since a single function can handle different type of objects at runtime. This significantly reduces the size of codebase, and it also makes the source code more readable.

Questions